

Basic Timing

1. using time and usr/bin/time to measure speed of matmul_slow.py

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ /usr/bin/time -v time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
0.21user 0.01system 0:00.23elapsed 99%CPU (0avgtext+0avgdata 13300maxresident)k
20inputs+0outputs (0major+1945minor)pagefaults 0swaps
    Command being timed: "time python3 matmul_slow.py"
    User time (seconds): 0.21
    System time (seconds): 0.01
    Percent of CPU this job got: 99%
    Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.23
    Average shared text size (kbytes): 0
    Average unshared data size (kbytes): 0
    Average stack size (kbytes): 0
    Average total size (kbytes): 0
    Maximum resident set size (kbytes): 13300
    Average resident set size (kbytes): 0
    Major (requiring I/O) page faults: 0
    Minor (reclaiming a frame) page faults: 2031
    Voluntary context switches: 7
    Involuntary context switches: 3
    Swaps: 0
    File system inputs: 20
    File system outputs: 0
    Socket messages sent: 0
    Socket messages received: 0
    Signals delivered: 0
    Page size (bytes): 4096
    Exit status: 0
ubuntu@ubuntu:~/CWM-FDI/assignment1$ time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

real    0m0.233s
user    0m0.224s
sys     0m0.009s
ubuntu@ubuntu:~/CWM-FDI/assignment1$
```

We can see that sys time is the lowest, then user, then real being the slowest.

These different metrics likely compare different speeds for different parts of the process.

- System time likely corresponds purely to time on the cpu (ie. time spent in the kernel)
- User time likely includes time for i/o, and external commands/ linking processes
- Real time could even include UI and frontend time in the terminal (Ie. full process - wall clock)

2.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ for i in 1 2 3 4 5; do /usr/bin/time -f "%e" python3
matmul_slow.py; done
n=128 reps=2 checksum=107389.290000
0.26
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
ubuntu@ubuntu:~/CWM-FDI/assignment1$
```

You would likely want to report the mean and standard deviation across many runs.

Reading Counters with perf stat

3.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -e cycles,instructions python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py':

      809203715      cycles
      2299005160      instructions          #    2.84  insn per cycle

      0.229883993 seconds time elapsed

      0.221159000 seconds user
      0.008005000 seconds sys
```

4.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py' (5 runs):

      809269959      cycles
( +-  0.47% )
      2295636230      instructions          #    2.84  insn per cycle
( +-  0.05% )
       584256      cache-misses
( +-  1.38% )

      0.22739 +- 0.00115 seconds time elapsed ( +-  0.50% )
```

We can see 2.86 instructions per cycle, which implies that more than 1 core is being used as a single core would only be able to achieve at most 1 instruction per cycle. (The CPU could also be using pipelining where before on instruction has finished executing in the CPU another is passed into the start – ie. multiple instructions can exist in the CPU execution “pipeline” at once.

We see 584256 cache misses, ie. 0.072% of cycles have a cache miss. Without a reference we do not know if this is low or high, however we can comment that cache hits improve efficiency, so fewer cache misses is advantageous to performance. This is because the cache is closer to the CPU and is faster memory, so the program can run faster if more data is pulled from the cache than from main memory.

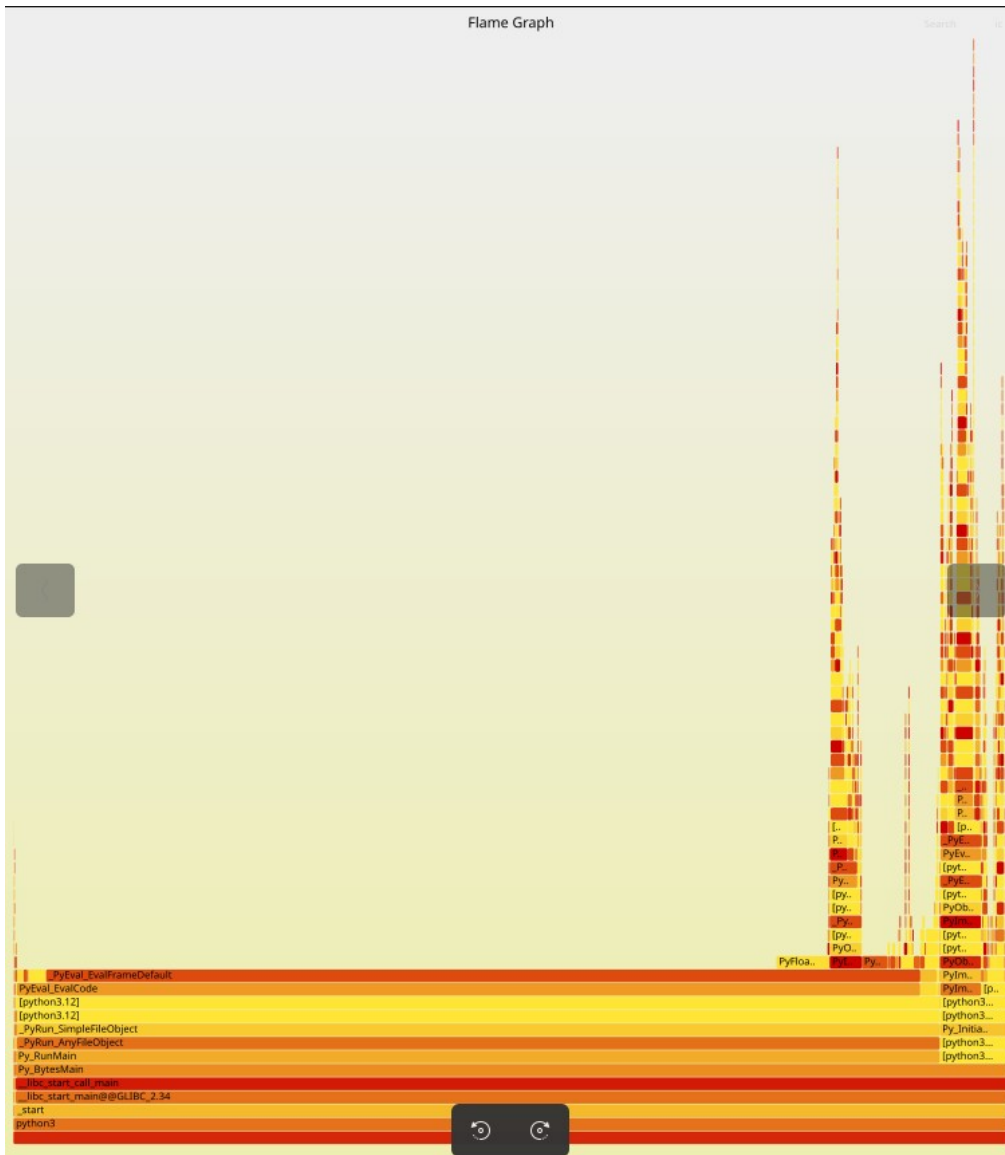
5.

Samples: 927 of event 'cycles:P', Event count (approx.): 814558161					
	Children	Self	Command	Shared Object	Symbol
+	99.82%	0.00%	python3	python3.12	[.] _start
+	99.76%	0.00%	python3	libc.so.6	[.] __libc_start_main@@GLIBC_2.34
+	99.76%	0.00%	python3	libc.so.6	[.] __libc_start_call_main
+	99.76%	0.00%	python3	python3.12	[.] Py_BytesMain
+	96.14%	0.00%	python3	python3.12	[.] PyEval_EvalCode
+	94.70%	75.02%	python3	python3.12	[.] _PyEval_EvalFrameDefault
+	92.86%	0.00%	python3	python3.12	[.] Py_RunMain
+	92.75%	0.00%	python3	python3.12	[.] _PyRun_AnyFileObject
+	92.75%	0.00%	python3	python3.12	[.] _PyRun_SimpleFileObject
+	90.90%	0.00%	python3	python3.12	[.] 0x000000000006b5263
+	90.79%	0.00%	python3	python3.12	[.] 0x00000000000608b52
+	8.55%	0.00%	python3	python3.12	[.] PyImport_ImportModuleLevelObject
+	8.55%	0.00%	python3	python3.12	[.] PyObject_CallMethodObjArgs
+	8.55%	0.00%	python3	python3.12	[.] 0x0000000000054a087
+	8.43%	0.00%	python3	python3.12	[.] 0x0000000000058206d
+	7.82%	0.00%	python3	python3.12	[.] 0x000000000005d36ac
+	6.90%	0.00%	python3	python3.12	[.] 0x000000000006bc965
+	6.83%	0.00%	python3	python3.12	[.] 0x000000000006bca35
+	6.83%	0.00%	python3	python3.12	[.] Py_InitializeFromConfig
+	6.33%	5.24%	python3	python3.12	[.] PyFloat_FromDouble
+	5.56%	0.00%	python3	python3.12	[.] 0x000000000005d39f4
+	5.55%	0.00%	python3	python3.12	[.] 0x000000000006b0bc4
+	5.21%	0.00%	python3	python3.12	[.] PyObject_Vectorcall
+	4.92%	0.00%	python3	python3.12	[.] PyImport_Import
+	4.92%	0.00%	python3	python3.12	[.] PyObject_CallFunction
+	4.81%	0.00%	python3	python3.12	[.] PyImport_ImportModule
+	3.99%	0.00%	python3	python3.12	[.] 0x000000000006b1132
+	3.06%	2.73%	python3	python3.12	[.] PyLong_FromLong
+	2.74%	0.00%	python3	python3.12	[.] _PyObject_MakeTpCall
+	2.55%	0.00%	python3	python3.12	[.] 0x000000000005d3759
+	2.55%	0.00%	python3	python3.12	[.] PyRun_StringFlags
+	2.53%	0.21%	python3	[kernel.kallsyms]	[k] entry_SYSCALL_64_after_hwframe
+	2.32%	0.00%	python3	[kernel.kallsyms]	[k] do_syscall_64
+	2.31%	0.00%	python3	[kernel.kallsyms]	[k] x64_sys_call
+	1.81%	0.00%	python3	python3.12	[.] 0x000000000005d2acb
+	1.80%	0.85%	python3	python3.12	[.] PyObject_Malloc
+	1.70%	0.00%	python3	python3.12	[.] 0x00000000000599c17
+	1.63%	0.00%	python3	python3.12	[.] 0x000000000006b542c
+	1.63%	0.00%	python3	python3.12	[.] 0x000000000006b55bd
+	1.63%	0.00%	python3	python3.12	[.] Py_Exit
+	1.63%	0.00%	python3	python3.12	[.] Py_FinalizeEx
+	1.27%	0.00%	python3	python3.12	[.] PyObject_VectorcallDict

The largest fraction of samples is consumed by `_PyEval_EvalFrameDefault`, likely corresponding to the evaluation of the matrix multiplication (the numerical operations).

Other high consumers are `PyFloat_FromDouble` (likely a type conversion call) and `PyLond_FromLong`, again a type conversion call.

6.



From the above, we can still see that `_PyEval_EvalFrameDefault` dominates runtime, which makes sense as the user readable output from the previous question just represents the input to the frame graph generation script.

From this, we can deduce that the likely bottlenecks in the `matmul_slow.py` program are the loops themselves in the functions.

7.

Matmul Fast1

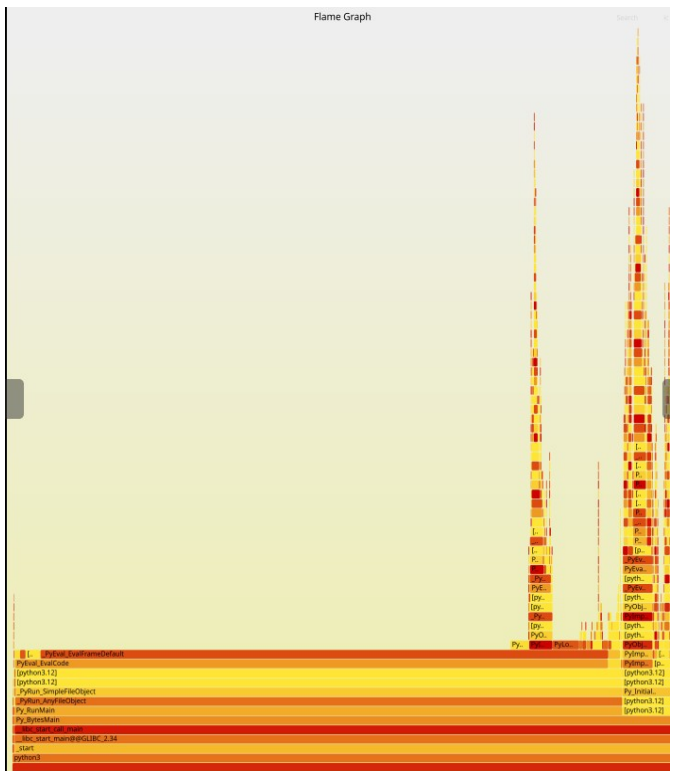
```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_fast.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_fast.py' (5 runs):

      755448223      cycles
    ( +- 2.36% )
      2059914584      instructions          #      2.73      insn per cycle
    ( +- 0.04% )
           604006      cache-misses
    ( +- 2.02% )

      0.21215 +- 0.00481 seconds time elapsed ( +- 2.27% )
```

slightly faster mean time, as the rows are initialized so we pull from the stack not the heap each iteration which exploits spatial and temporal locality making data retrieval faster.



Matmul Fast2

```

ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-miss
s python3 matmul_fast.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_fast.py' (5 runs):

    756198134      cycles
    ( +- 0.15% )
    2273207637    instructions          #      3.01  insn per cycle
    ( +- 0.04% )
    595107        cache-misses
    ( +- 1.55% )

    0.212379 +- 0.000960 seconds time elapsed ( +- 0.45% )

```

loops reordered to reduce the number of inner loops. We get a larger number of instructions per cycle, suggesting that this format allows for better pipelining.

